Adaptive Resource Allocation for Mobile Edge Computing in Internet of Vehicles: A Deep Reinforcement Learning Approach

Junhui Zhao, *Senior Member, IEEE*, Haoyu Quan, Minghua Xia, *Senior Member, IEEE*, and Dongming Wang, *Member, IEEE*

***Abstract*—Mobile edge computing (MEC) has emerged in recent years as an effective solution to the challenge of limited vehicle resources in the Internet of Vehicles (IoV), especially for computation-intensive vehicle tasks. This paper investigates a multi-user MEC system with an active task model in high-dynamic IoV scenarios. To improve the MEC performance regarding system capacity, task service delay, and energy consumption, we design an adaptive joint resource allocation scheme based on deep reinforcement learning (DRL), which includes uplink, computing, and downlink resource allocation. Further, a multi-actor parallel twin delayed deep deterministic policy gradient (MAPTD3) algorithm is devised to jointly and adaptively optimize these strategies during each time slot. Finally, numerical results demonstrate that the proposed adaptive joint resource allocation scheme improves system performance significantly while satisfying task delay and system resource constraints. In addition, the space complexity of the designed optimization algorithm is lower than that of conventional DRL algorithms.**

***Index Terms*—Mobile edge computing (MEC), Internet of Vehicles (IoV), adaptive joint resource allocation, deep reinforcement learning (DRL).**

Manuscript received 19 May 2023; revised 22 September 2023; accepted 16 November 2023. Date of publication 28 November 2023; date of current version 22 April 2024. This work was supported in part by the National Natural Science Foundation of China under Grants U2001213 and 61971191, in part by Beijing Natural Science Foundation under Grant L201011, in part by the National Key Research and Development Project of China under Grant 2020YFB1807204, in part by the Jiangxi Key Laboratory of Artificial Intelligence Transportation Information Transmission and Processing under Grant 20202BCD42010, and in part by Jiangxi Graduate Student Innovation Foundation under Grant YC2022-s496. The review of this article was coordinated by Dr. Zilong Liu. *(Corresponding author: Junhui Zhao.)*

Junhui Zhao is with the School of Information Engineering, East China Jiaotong University, Nanchang 330013, China, and also with the School of Electronic and Information Engineering, Beijing Jiaotong University, Beijing 100044, China (e-mail: junhuizhao@hotmail.com).

Haoyu Quan is with the School of Information Engineering, East China Jiaotong University, Nanchang 330013, China (e-mail: koterial@hotmail.com).

Minghua Xia is with the School of Electronics and Information Technology, Sun Yat-sen University, Guangzhou 510006, China, and also with the Southern Marine Science and Engineering Guangdong Laboratory, Sun Yat-sen University, Zhuhai 519082, China (e-mail: xiamingh@mail.sysu.edu.cn).

Dongming Wang is with the National Mobile Communications Research Laboratory, Southeast University, Nanjing 210096, China, and also with the Purple Mountain Laboratories, Nanjing 211111, China (e-mail: wangdm@seu.edu.cn).

Digital Object Identifier 10.1109/TVT.2023.3335663

I. Introduction

With the broad development of the Internet of Vehicles (IoV) technology in the 5G era, various in-vehicle applications have emerged in real life, such as speech recognition, street maps, and autonomous driving, which are used to efficiently assist drivers during the travel [1], [2], [3]. These applications are usually computationally intensive and have strict delay requirements. Running on vehicles with limited computing resources will consume plenty of time and energy, and it is hard to achieve satisfactory performance. To overcome these challenges, mobile edge computing (MEC) systems with high computing capability are considered a promising solution, through which vehicles can enhance their computing capabilities by connecting to nearby base stations (BS) and accessing relevant hardware [4].

MEC systems depend highly on wireless communications in real-world applications, bringing additional communication delay and energy consumption to applications [5]. Thus, a more careful trade-off between communication and computing costs must be accounted for. In addition, unlike traditional centralized cloud computing systems, MEC systems are distributed at the edge of the network, which is closer to users and has a lower communication delay. But this also means that the system communication and computing resources are finite [6]. Meanwhile, distributed deployment implies that MEC systems require higher operation costs. Improving the existing system's capacity to reduce user cost is also a severe challenge [7]. Hence, it is necessary to design resource allocation schemes that can efficiently manage the various resources of MEC systems while balancing system capacity, service delay, and energy consumption performance.

Traditional optimization algorithms, such as convex optimization and game theory, are widely used to optimize the resource allocation strategy of MEC systems [8], [9], [10]. These algorithms usually require multiple iterations and suffer from exponentially increasing solution space complexity, especially in large-scale IoV scenarios [5], [11]. Therefore, adopting the deep reinforcement learning (DRL) algorithm to optimize resource allocation strategy has attracted increasing attention from researchers because of the excellent decision-making performance [12], [13], [14]. Moreover, the high-dynamic characteristics of IoV scenarios further complicate the resource allocation

0018-9545 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

problem. Such as the burst nature of vehicle access and task arrival will cause MEC systems to be more unpredictable [15]. In this case, the fixed resource allocation scheme is hard to improve the system resource utilization. To cope with unexpected vehicle access and task arrival, adaptive resource allocation schemes that can be adjusted in real-time with the system state have been widely studied [5], [16]. Most works in the literature have generally assumed available prior information about the time of vehicle access and task arrival. Unfortunately, it is hard to obtain vehicle access and task arrival time in practice, and the above assumptions will weaken the generalization ability of the scheme. Therefore, training an agent without prior information about the environment becomes challenging.

To resolve the challenges mentioned above, this paper designs an adaptive multi-resource joint allocation scheme using a DRL approach, which adaptively makes joint allocation of uplink, computing, and downlink resources in IoV scenarios efficient and economical. In brief, the major contributions of this paper are summarized as follows:

- An adaptive joint resource allocation scheme is proposed to decouple the multi-resource coordinated allocation strategy into three sub-strategies: uplink, computing, and downlink resource allocation. Adjusting these sub-strategies in real-time can optimize the system performance regarding capacity, service delay, and energy consumption.
- A multi-actor parallel twin delayed deep deterministic policy gradient (MAPTD3) algorithm is developed for the proposed scheme, and the task-slicing method is adopted to define the agent's state space and reward function. This design enables the agent to learn an efficient allocation strategy without prior knowledge or assumptions about the environment and realize the adaptive resource allocation in a continuous domain.
- Performance analysis and extensive simulation results demonstrate that the proposed adaptive scheme outperforms the conventional fixed scheme. In addition, optimizing the system through the MAPTD3 algorithm can significantly reduce the space complexity of the resource allocation model in complex scenarios while effectively improving the system's performance.

To elaborate on the above contributions, the rest of this paper is organized as follows. Section II reviews the related works. Section III discusses the system model and problem formulation. The proposed adaptive joint resource allocation scheme and MAPTD3 algorithm are elaborated in Section IV. Simulation results are presented and discussed in Section V, and finally, Section VI concludes this paper.

II. Related Works

Many efforts have been devoted to optimizing resource allocation strategies to maximize the system performance with limited system resources. According to the optimization method, these schemes can be divided into optimization-based and machine learning-based schemes, as each described below.

A. Optimization-Based Schemes

The resource allocation optimization problem in MEC systems can be formulated as a mixed integer nonconvex problem. The standard solution is to decouple the problem into multiple sub-problems and use multiple sub-algorithms to obtain the optimal solution through mutual iterations. For example, the work [8] studied how to improve the energy efficiency of MEC systems in Internet of Things scenarios. The energy efficiency is effectively improved by decoupling the problem into separate communication and computing resource allocation problems and using a matching algorithm and sequential convex programming algorithm for iterative solutions. The contribution [9] studied the joint optimization problem of computation offloading and resource allocation in a vehicle-edge collaborative MEC systems, where a distributed algorithm was devised to decouple the problem into two sub-problems. By using the Lagrange multiplier method, the utilization of system resources was effectively improved. The study [10] decoupled the optimization problem into two sub-problems according to the characteristics of NP-hard, and the scheme of iteratively solving the joint optimization problem using the Hungarian algorithm and the graph coloring algorithm was designed. However, the methods mentioned above are all based on a fixed resource allocation strategy. They require multiple iterations, so it is hard to achieve satisfactory real-time decision performance in high-dynamic MEC systems, such as IoV.

Lyapunov-based dynamic resource allocation schemes have received extensive attention to resolve the challenges imposed by high-dynamic MEC systems. For example, the work [17] divided the optimization problem into several sub-problems in each time slot by minimizing the derived upper bound of the Lyapunov drift-plus-penalty function, and the dynamic adjustment of communication and computing resources allocation was realized. In [3], the authors proposed a two-timescale edge computing framework based on the Lyapunov optimization algorithm. This effectively solved the short-term resource allocation strategy problem for dynamic MEC systems. However, Lyapunov-based schemes usually require system assumptions (e.g., times of task arrival) or prior information (e.g., statistics of average task arrival rate) [15], [18], [19] to ensure the stability of the task queue, which is not always feasible in practical applications.

B. Machine Learning-Based Schemes

Considering machine learning technology's excellent performance in control and decision-making in recent years, more and more researchers are trying to adopt machine learning technology to optimize resource allocation problems in MEC systems. For example, the work [20] studied the association problem between multiple unmanned aerial vehicles (UAV) and proposed a reinforcement learning (RL)-based UAV resource allocation scheme, which verified that the RL-based scheme could achieve the optimal solution. The contribution [12] studied the stochastic traffic flow conditions of the IoV and designed an offloading scheme in a vehicle edge computing network. By transforming the problem into a semi-Markov process and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

using the Q-learning algorithm to optimize the communication resource allocation strategy, the total system utility of the MEC system is maximized. However, state parameters in IoV scenarios are usually continuous, such as the communication distance between the vehicle and the BS and task data size. Therefore, an even more severe challenge is to define the continuous state space only relying on the discrete Q-table in the Q-learning algorithm.

From the neural networks' powerful fitting ability, DRL algorithms can better handle continuous state spaces. The schemes designed in [13], [21], [22] all adopt the deep Q-network (DQN) algorithm to optimize the resource allocation strategy. In [21], the authors verified the advantages of the DQN algorithm compared with the Q-learning algorithm by comparing the total cost in different scenarios. The authors of [22] proposed to define the reward function in the form of entropy weight and used the DQN algorithm to optimize the resource allocation strategy, effectively improving the learning efficiency of the agent. The work [13] studied the problem of spectrum resource allocation in train-to-train scenarios and proposed the distributed multi-agent DQN algorithm to realize the user-autonomous channel resource selection. The results show that the distributed scheme can effectively improve the communication success rate. Notably, the DQN algorithm only uses a single neural network to fit the state space while the action space is still discrete. Therefore, the above schemes can only optimize discrete strategy, and the resource needs to be pre-discretized. However, the degree of discretization is often challenging to determine, requiring a trade-off between system performance and model complexity.

To further improve the performance of resource allocation strategy, DRL algorithms that can optimize continuous strategies are favored by more and more researchers. The research [23] designed an online resource allocation scheme based on the deep deterministic policy gradient (DDPG) algorithm, which can allocate computing resources more efficiently and fairly. The study [24] developed a CPU cycle frequency-aided DDPG scheme for resource management and optimization for large-scale MEC systems. The work [14] studied the resource allocation problem in UAV-assisted vehicles accessing MEC system, and then applied the multi-agent DDPG algorithm to formulate an individual optimization strategy for each edge server and realized the distributed decision-making of the MEC system on vehicle association and resource allocation. Although the above schemes improve the performance of resource allocation strategy, it also faces the fact that the DDPG algorithm is unstable during the training process, which is unacceptable for IoV systems with high security requirements [25]. To overcome this problem, the authors of [5] proposed using the temporal feature extraction network to improve the DDPG algorithm. This improvement effectively enhanced the robustness of the agent training process in high-dynamic MEC systems but ignored the problem of model complexity and could only realize the allocation of uplink bandwidth. At the same time, the above researches generally require prior knowledge about the environment (e.g., rules of vehicle access), and the training process is only partially model-free.

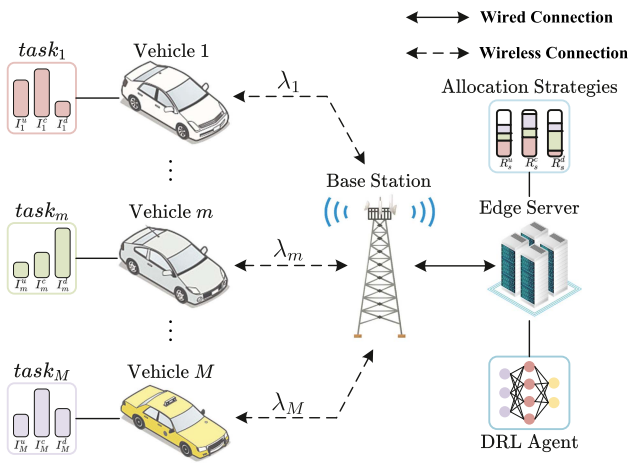


Fig. 1. Illustrative MEC model in a high-dynamic IoV network.

In contrast to the above research, this paper designs an adaptive joint resource allocation scheme and customizes the MAPTD3 agent to be suitable for multiple strategies joint optimization in high-dynamic IoV systems. In particular, we eliminate the dependence on prior information about the environment during the training process and effectively reduce the space complexity of the adaptive resource allocation model while ensuring system performance.

III. System Model and Problem Formulation

This section starts with the MEC system and dynamic task models, then discusses the service delay and energy consumption in the communication and computing models. Finally, we formulate the optimization problem for resource allocation.

A. System Model

Fig. 1 shows a multi-user MEC system in an IoV application, consisting of a BS equipped with a massive multiple-input multiple-output (MIMO) antenna array, an edge server with high-performance computing capabilities and M vehicles, each performing an independent task. Among them, the communication and computing resources are limited, without losing generality. Specifically, the communication resources include uplink bandwidth and downlink bandwidth, and their specific values depend on the amount of spectrum resource owned by the BS operator. In contrast, computing resources refer to the computing capability of the inherent CPU, and the particular value depends on the CPU configured by the edge server. Accordingly, we define a 3-tuple set (R^u, R^c, R^d) to represent the MEC system resources, where R^u and R^d denote the uplink and downlink communication resources owned by the BS, respectively, and R^c represents the computing resources owned by the edge server. Table I summarizes the necessary notations used in the paper for ease of tractability.

1) Dynamic Task Model: We adopt a stochastic dynamic task model to reflect the characteristics of the high-dynamic IoV MEC system. Fig. 2 illustrates the distributions of various

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

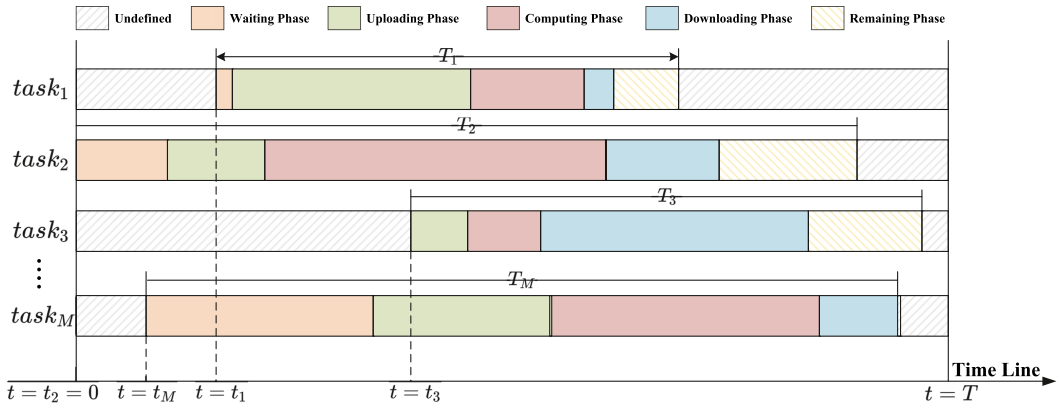


Fig. 2. Illustration of the dynamic task model comprising five phases.

TABLE I
NOTATION DEFINITIONS

Symbol	Description
m, M	vehicle index/set
t, T, τ	time index/set/slot length
R^u, R^c, R^d	uplink/computing/downlink resources of the MEC system
R^u_s, R^c_s, R^d_s	uplink/computing/downlink resource allocation strategy
R^u_m, R^c_m, R^d_m	uplink/computing/downlink resource allocated to vehicle m
ρ^u_m, ρ^d_m	uplink/downlink SNR of vehicle m
v^u_m, v^d_m	uplink/downlink Tx rate of vehicle m
P^t_{bm}, P^r_{bm}	Tx/Rx power of BS to/from vehicle m
P^t_m, P^r_m, P^0_m	Tx/Rx/standby power of vehicle m
λ_m, t_m, T_m	priority weight/arrival time/delay threshold of $task_m$
I^u_m, I^c_m, I^d_m	initial upload/computing/download data size of $task_m$
D^u_m, D^c_m, D^d_m	remaining upload/computing/download data size of $task_m$
x^u_m, x^c_m, x^d_m	upload/computing/download phase identifier of $task_m$
$d^{\text{wait}}_m, d^{\text{comm}}_m, d^{\text{comp}}_m, d_m$	waiting/communication/computing/service delay of vehicle m
$e^{\text{comm}}_m, e^0_m, e_m$	Tx/standby/total energy consumption of vehicle m
$\theta^c, \theta^t, \theta^e$	capacity/delay/energy metrics weight factor
c_m	overall performance of vehicle m
N_0	power spectral density of AWGN

tasks in the time domain. It is seen that the task arrival time and delay threshold of multiple tasks are highly random, and different tasks have different requirements for system resources. For the $task_m$ performed by vehicle m, we define a 6-tuple set $(t_m, T_m, I^u_m, I^c_m, I^d_m, \lambda_m)$, where t_m denotes the arrival time and T_m the delay threshold; I^u_m, I^c_m and I^d_m denotes the initial uploading, computing, and downloading data size, respectively; and λ_m is the priority weight. Meanwhile, since each task will execute uploading, computing, and downloading phases in sequence, three binary control variables (x^u_m, x^c_m, x^d_m) are introduced to indicate the current phase of $task_m$, where $x^u_m = 1, x^c_m = 1$ and $x^d_m = 1$ represent the uploading, computing and downloading phase of the task, respectively, and $x^u_m = x^c_m = x^d_m = 0$ otherwise. In addition, considering the waiting time generated during the task offloading process and the possibility of tasks completed ahead of schedule, the definition of arrived tasks in the time domain is divided into five phases: the waiting phase represents the offloading request response time d^{wait}_m; the uploading, computing, and downloading phases represent the processing flow, and finally, the remaining phase corresponds to the time completed in advance.

It is assumed the communication and computing resources of the MEC system can be allocated independently. To better capture the system dynamics, the proposed adaptive joint resource allocation is accurate to every time slot, where the length of each time slot is set to τ, with time slot index t.

2) Service Delay Model: The service delay of tasks in the MEC system mainly comprises the waiting delay of request response, the communication delay between the vehicle and the BS, and the computing delay of the edge server processing tasks. Next, we analyze the last two delays.

The *communication delay* between the vehicle and the BS is affected by the amount of communication bandwidth allocated to the vehicle and the data size of the task being uploaded and downloaded. To accurately model real-world IoV scenarios, the delay caused by task result return is considered. In addition, the wireless connection between the vehicle and the BS brings inevitable communication delays and is disturbed by environmental noise. On the other hand, the communication delay between the edge server and the BS is negligible because of their high-throughput wireline connection [5]. Here, we define $R^u_s(t) \triangleq \{R^u_m(t)\}_M$ to represent the uplink bandwidth allocation strategy at time slot t, and $R^d_s(t) \triangleq \{R^d_m(t)\}_M$ to indicate the downlink one, in which $R^u_m(t)$ and $R^d_m(t)$ denote the uplink and downlink bandwidth allocated to vehicle m at time slot t, respectively. As the communication resources of BS are limited, the communication resource allocation strategy is constrained by $\sum_{m\in M} R^u_m(t) \le R^u, \forall t$, and $\sum_{m\in M} R^d_m(t) \le R^d, \forall t$. Assuming the communication spectrum resources allocated to all vehicles are mutually orthogonal, the co-channel interference between different vehicles is negligible.

Next, at time slot t, the uplink signal-to-noise ratio (SNR) between vehicle m and the BS is defined as

$$\rho^u_m(t) = \frac{P^r_{bm}}{R^u_m(t) N_0}, \tag{1}$$

where N_0 denotes the power spectral density of additive white Gaussian noise (AWGN), and P^r_{bm} is the signal power received by the BS from vehicle m. By recalling the seminal Shannon's formula, the uplink transmission rate of vehicle m at time slot t

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

can be expressed as

$$\upsilon_m^u(t) = R_m^u(t)\log_2\left(1+\rho_m^u(t)\right). \tag{2}$$

Similarly, the downlink SNR and data rate of vehicle m at time slot t can be expressed as

$$\rho_m^d(t) = \frac{P_m^r}{R_m^d(t)N_0}, \tag{3}$$

$$\upsilon_m^d(t) = R_m^d(t)\log_2\left(1+\rho_m^d(t)\right), \tag{4}$$

respectively, where P_m^r is the signal power received by vehicle m from the BS. Given the communication resource allocation strategy, the size of the remaining communication data at the next time slot can be computed as

$$D_m^u(t+1) = D_m^u(t) - x_m^u\tau\upsilon_m^u(t) = I_m^u - \sum_{i=0}^{t} x_m^u\tau\upsilon_m^u(i), \tag{5}$$

$$D_m^d(t+1) = D_m^d(t) - x_m^d\tau\upsilon_m^d(t) = I_m^d - \sum_{i=0}^{t} x_m^d\tau\upsilon_m^d(i), \tag{6}$$

where $D_m^u(t)$ and $D_m^d(t)$ denote the remaining upload and download data size of $task_m$ at time slot t, respectively. Then, the communication delay d_m^{comm} can be obtained as the solution to $D_m^u(t)=0$ and $D_m^d(t)=0$. It is noteworthy that the domain of $D_m^u(t)$ and $D_m^d(t)$ are both $[t_m, t_m+T_m]$, and satisfies $D_m^u(t_m)=I_m^u$ and $D_m^d(t_m)=I_m^d$. Define $D_m^{u(-1)}(t)$ and $D_m^{d(-1)}(t)$ being the inverse functions of $D_m^u(t)$ and $D_m^d(t)$, respectively, then the communication delay of vehicle m can be expressed as

$$d_m^{\text{comm}} = D_m^{u(-1)}(0) + D_m^{d(-1)}(0). \tag{7}$$

On the other hand, the *computing delay* for edge computing depends on the computing resources allocated to the vehicle and the number of CPU cycles required by the task. Let $R_s^c(t) \triangleq \{R_m^c(t)\}_M$ be the computing resource allocation strategy at time slot t. Like the communication resource allocation strategy described above, the computing resource allocation is also constrained by $\sum_{m\in M} R_m^c(t) \le R^c, \forall t$. Given the computing resource allocation strategy, the size of the remaining computing data at the next time slot can be computed as

$$D_m^c(t+1) = D_m^c(t) - x_m^c\tau R_m^c(t) = I_m^c - \sum_{i=0}^{t} x_m^c\tau\upsilon_m^c(i). \tag{8}$$

Then, like (7), the computing delay of vehicle m can be expressed as

$$d_m^{\text{comp}} = D_m^{c(-1)}(0). \tag{9}$$

Finally, by accounting for the waiting delay d_m^{wait}, the communication delay d_m^{comm}, and the computing delay d_m^{comp}, the service delay d_m of vehicle m can be expressed as

$$d_m = d_m^{\text{wait}} + d_m^{\text{comp}} + d_m^{\text{comm}}. \tag{10}$$

3) Energy Consumption Model: For vehicle m, the energy consumption of MEC mainly comes from the consumed energy e_m^{comm} in the uploading phase, and the standby energy consumption e_m^0 in all service phases [26]. Assume that the standby power of vehicle m is fixed as P_m^0 and the transmit power is as constant as P_m^t, the total energy consumption of vehicle m can be computed as

$$e_m = e_m^{\text{comm}} + e_m^0 = D_m^{u(-1)}(0)P_m^t + d_m P_m^0. \tag{11}$$

B. Problem Formulation

We aim to improve the system capacity while reducing task service delay and vehicle energy consumption by adjusting the allocation strategy of communication and computing resources during each time slot. Therefore, it is necessary to measure the overall performance of each vehicle, defined for vehicle m as

$$c_m \triangleq \theta^c x_m - \theta^t d_m - \theta^e e_m, \tag{12}$$

where θ^c, θ^t, and θ^e denote the weight factors of the system capacity, service delay, and energy consumption, respectively. Moreover, x_m in (12) is a binary variable and $x_m=1$ means that $task_m$ is completed within the delay threshold, and $x_m=0$ otherwise. Finally, for all $t\in T$ and $m\in M$, the optimal adaptive resource allocation problem can be formulated as,

$$\mathcal{P}1: \max_{R_s^u(t),R_s^c(t),R_s^d(t)} \sum_{m\in M}\lambda_m c_m, \tag{13a}$$

$$\text{s.t.}\quad \text{C1: } x_m^i = \{0,1\},\ \forall i\in\{u,c,d\}, \tag{13b}$$

$$\text{C2: } x_m^u + x_m^c + x_m^d \le 1, \tag{13c}$$

$$\text{C3: } 0 \le D_m^u(t) \le I_m^u,\ \forall t\in[t_m, t_m+T_m], \tag{13d}$$

$$\text{C4: } 0 \le D_m^c(t) \le I_m^d,\ \forall t\in[t_m, t_m+T_m], \tag{13e}$$

$$\text{C5: } 0 \le D_m^d(t) \le I_m^d,\ \forall t\in[t_m, t_m+T_m], \tag{13f}$$

$$\text{C6: } 0 \le R_m^u(t) \le R^u, \tag{13g}$$

$$\text{C7: } 0 \le R_m^d(t) \le R^d, \tag{13h}$$

$$\text{C8: } 0 \le R_m^c(t) \le R^c, \tag{13i}$$

$$\text{C9: } \sum_{m\in M} R_m^u(t) \le R^u, \tag{13j}$$

$$\text{C10: } \sum_{m\in M} R_m^d(t) \le R^d, \tag{13k}$$

$$\text{C11: } \sum_{m\in M} R_m^c(t) \le R^c, \tag{13l}$$

where the constraints C1–C2 limit the processing flow of tasks, C3–C5 give the task delay thresholds and the ones C6–C11 indicate the system resource allocation strategies. More specifically, C1–C2 state that the task can only be in one phase at the same time; C3–C5 show that the task is only defined in $[t_m, t_m+T_m]$; C6–C8 specify that the resource allocation ratio for each vehicle varies between 0 and 1, and the constraints C9–C11 mean that the total resources occupied by all vehicles cannot exceed the resources owned by the MEC system.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

It is clear that $\mathcal{P}1$ is a classic mixed-integer nonlinear optimization problem, which is NP-hard [15]. An enumeration resource allocation strategy can be applied to solve this problem. However, since all alternative resource allocation strategies must be exhaustively searched, the computational complexity is too high to be accepted in practice. Moreover, prior information about the IoV MEC environment cannot be obtained in reality. To address these issues, the following section proposes an adaptive decision-making method through online learning.

IV. Adaptive Joint Resource Allocation Scheme

In this section, an adaptive joint resource allocation scheme is devised to solve $\mathcal{P}1$, which mainly consists of three unique designs: *i)* A task-slicing method is developed to describe the state space and reward function of the agent; *ii)* A dynamic task queue is constructed to solve the vehicle random access problem; *iii)* A MAPTD3 algorithm is designed to optimize multiple strategies jointly. This section is divided into five parts for better readability: the agent design, dynamic task queue, TD3 algorithm, MAPTD3 algorithm, and the adaptive resource allocation framework.

A. Agent Design

In principle, the goal of the DRL agent is to maximize the long-term cumulative reward by dynamically adjusting the actions in different states, which is consistent with the objective function $\mathcal{P}1$. Therefore, it is possible to optimize the adaptive resource allocation scheme of the system by using the DRL algorithm. Like previous studies based on DRL algorithms [12], [13], [14], [21], [22], [23], [24], the proposed MAPTD3 algorithm has three essential elements: state space, action space, and reward function, but the difference lies that the task-slicing method is adopted to design the state space and reward function. With this method, even without prior information or assumptions about the IoV MEC environment, the resource allocation strategy derived by the MAPTD3 agent can be arbitrarily close to the optimal solution, and the training process is model-free. Next, we design the state space, action space, and reward function in sequence.

1) State Space: The state information of the agent comes from observing the running tasks in the MEC system. The state of $task_m$ at time slot t contains seven parameters: the time state s_m^T, the upload data state s_m^u, the computing data state s_m^c, the download data state s_m^d, the uplink power state s_m^{p1}, the downlink power state s_m^{p2}, and the priority weight state s_m^w. Among them, the definition of the data state (s_m^u, s_m^c, s_m^d) adopts the method of normalizing the remaining data size, expressed as

$$s_m^i(t) = \frac{D_m^i(t)}{D_{\text{norm}}^i}, \quad i \in \{u, c, d\}, \tag{14}$$

where D_{norm}^i is the normalized value, generally set as the upper limit of the initial data size of the task model in all train episodes. Similarly, the time state s_m^T is defined by the normalized remaining executable time:

$$s_m^T(t) = \frac{t_m + T_m - t}{T_{\text{norm}}}. \tag{15}$$

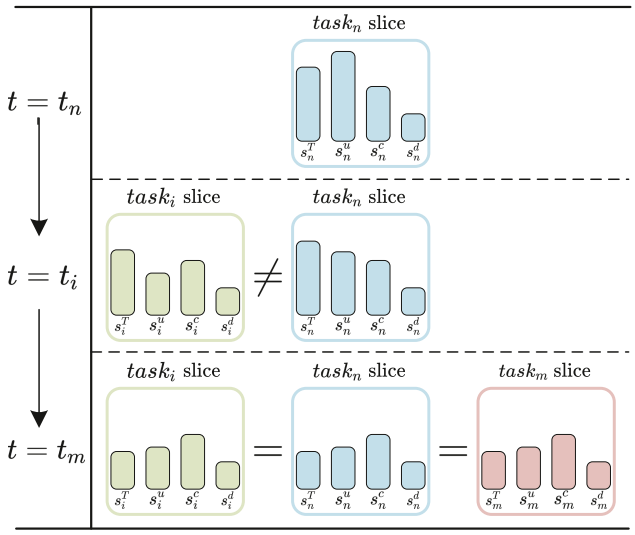


Fig. 3. Illustration of the task-slicing principle.

Fig. 3 illustrates the principle of task-slicing. It is seen that although the stochastic dynamic task model adopts different parameters to initialize these tasks, the slice initialized by $task_m$ at time slot t_m is equivalent to the running slices of $task_i$ and $task_n$. This means that each slice of a running task in the dynamic task model can be viewed as a newly arrived task, and both have the same subsequent decision chain when external conditions remain consistent, which strictly conforms to the definition of the Markov process about state transition: the future is only about the present, not history [27]. Therefore, it is possible to adjust the arrival times of different tasks to the same value (e.g., the earliest arrival time) in the training environment, given that the desired strategy is included in the trained ones. This approach skillfully eliminates the dependence on prior information (e.g., vehicle access and task arrival time) and dramatically improves the utilization of state experience during the training process. Moreover, the normalized received power is used to represent the power state (s_m^{p1}, s_m^{p2}), which not only facilitates the agent to obtain information quickly but also allows the transmit power and distance to take more possible values.

2) Action Space: As shown in Fig. 1, since the MEC system contains three kinds of resources, the agent needs to output three strategies to each allocate uplink bandwidth, computing resource, and downlink bandwidth. Accordingly, we define a 3-tuple set $(a_m^u(t), a_m^c(t), a_m^d(t))$ to represent the resource allocation strategy adopted by the agent for $task_m$ at time slot t:

$$a_m^i(t) = \frac{R_m^i(t)}{R^i}, \quad i \in \{u, c, d\}. \tag{16}$$

Compared to the state space, since the resource allocation strategy is constrained by (13), the action space of the agent is necessary to perform softmax normalization on the output of various resource allocation strategies [28]. However, the softmax normalization will always cause the allocation proportion of all resources to be 100%, which cannot be finely adjusted according

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

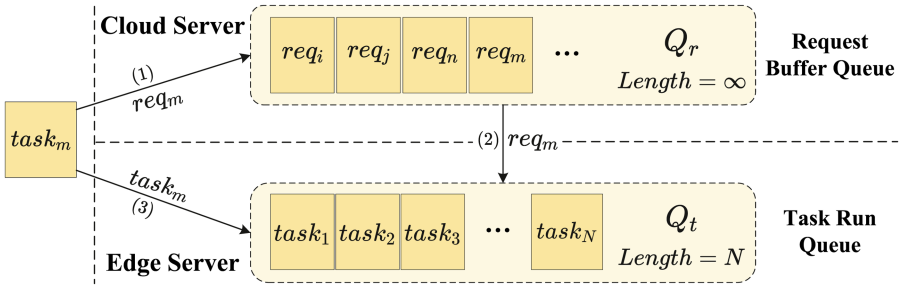


Fig. 4. Illustration of the dynamic task queue scheduling process.

to the system state. For example, when all vehicles in the MEC system are performing data communications, the computing resources of the edge server are still fully allocated instead of choosing the more reasonable hibernation strategy. Therefore, we introduce an additional action behind the actor network's action space to represent the proportion of unallocated resources, which is used to eliminate the effect of softmax normalization.

3) Reward Function: As the long-term goal of the agent needs to optimize, the design of the reward function is directly related to the final performance. To satisfy the optimization problem (13) while avoiding the problem of reward sparsity [29], the reward function needs to be defined for each time slot. Accordingly, the reward function $r(t)$ is defined as the weighted sum of all task slice rewards $r_m(t)$:

$$r(t) = \sum_{m \in M} s_m^w \, r_m(t), \tag{17}$$

where s_m^w denotes the priority weight of the the task slice reward function $r_m(t)$, computed by

$$r_m(t) = \theta^c r_m^C(t) - \theta^t r_m^T(t) r_m^D(t) - \theta^e r_m^E(t), \tag{18}$$

in which $r_m^C(t) = 1$ if $task_m$ is completed within the delay threshold and $r_m^C(t) = 0$ otherwise; $r_m^T(t) = 2 - s_m^T(t)$ and $r_m^D(t) = (s_m^u(t) + s_m^c(t) + s_m^d(t))/3$ denote the feedback of time and data size generated by $task_m$ slice, respectively; $r_m^E(t)$ denote the energy consumed in the current time slot, expressed as

$$r_m^E(t) = \begin{cases} \tau P_m^0, & \text{if } s_m^u(t) = 0; \\ \tau(P_m^t + P_m^0), & \text{otherwise.} \end{cases} \tag{19}$$

It is noteworthy that in (18), the product of $r_m^T(t)$ and $r_m^D(t)$ is used to jointly construct the reward for task slice size, which can generate more vital feedback for task slices that have either too short executable time or too large remaining data size in each time slot. Through this method, the agent will be forced to reduce the accumulation of task slices in both time and data size dimensions and ultimately achieve a reduction in service delay. At the same time, the time state s_m^T refers to the remaining executable time, while the time feedback r_m^T is related to the time that has been executed. Therefore, there is a complementary relationship between them. Moreover, the conventional expression $r_m^T(t) = 1 - s_m^T(t)$ makes the time feedback ratio between the later and early stages infinite, which could cause the agent to ignore optimization in the task early stage, so we adopt the deviation 2 in the definition of $r_m^T(t)$ (i.e., $r_m^T(t) = 2 - s_m^T(t)$ shown immediately after (18)) to balance the time feedback.

B. Dynamic Task Queue

Fig. 4 illustrates the dynamic task queue scheduling process applied in our model. Clearly, the MEC system has two queues: Q_r is the request buffer queue, which is used to cache the task offloading requests that have arrived but have not been processed, and is usually deployed on centralized devices (such as cloud server). Q_t is the task run queue, which records the tasks running in the current time slot and is generally deployed on distributed devices (such as edge servers). Considering the constraints of the parallel control capability of the agent and the unpredictability of the number of parallel tasks, the cache sizes of the Q_r are set to ∞, and Q_t is set to N.

As aforementioned, the number of parallel tasks in the MEC system is unpredictable, which makes the dimension of the neural network of the agent uncertain. Therefore, each task's offloading request will not be immediately responded to by Q_t. Instead, it will be transferred to Q_r for caching and then dynamically added to Q_t according to the priority weight, task slice size, and cache situation. Specifically, if Q_t has enough capacity to process a new task at time slot t, the task offloading request with the most considerable offload weight will be selected from Q_r and transmitted to Q_t first. Then, the corresponding task will be offloaded to the Q_t for processing. Otherwise, Q_r will decide to reserve the offloading request to the next time slot according to the delay threshold of the task or not. The value of Q_t changes dynamically over time for MEC systems. Once a task is completed or exceeds the delay threshold, it will be removed from Q_t at the end of the current time slot, and new tasks will be added dynamically at the beginning of the next time slot. To sum up, the dynamic task queue scheduling algorithm is summarized in Algorithm 1.

C. TD3 Algorithm

Although the DDPG algorithm can solve the problem in continuous state and action space, it leads to weak robustness and complex convergence during training. As a result, the twin delayed deep deterministic (TD3) algorithm was proposed to improve the agent's robustness [30]. Like DDPG, the TD3 algorithm is also based on the critic-actor structure, but three main differences exist between them, as each described below.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Dynamic Task Queue Scheduling Algorithm.

Initialization: Simulation time parameter T, BS and edge server parameter (R^u, R^c, R^d), vehicle set $m = \{1, 2, \ldots, M\}$, $task_m$ parameter $(t_m, T_m, I_m^u, I_m^c, I_m^d, \lambda_m)$, Q_r length $+\infty$ and Q_t length N.
for $t = 1$ to T **do**
  for $i = 1$ to M **do**
    if $t = t_i$ **then**
      Add req_i into Q_r;
    end if
  end for
  Recalculate the weight of all task offloading requests according to (17)-(18);
  whilelen$(Q_t) < N$ and len$(Q_t) > 0$
    Select req_i from Q_r with the maximum task offloading request weight;
    if $t_i \le t < t_i + T_i$
      Add $task_i$ into Q_t and remove req_i from Q_r;
    else
      Remove req_i from Q_r;
    end if
  end while
  Obtain state s from Q_t and generate corresponding action a by the DRL agent;
  Execute the resource allocation strategy according to a and run the system for a time slot;
  for $task_i$ in Q_t **do**
    if $t \ge t_i + T_i$ or $task_i$ finish **then**
      Remove $task_i$ from Q_t;
    end if
  end for
end for

1) Clipped Double Q-Learning: In the Markov decision process shown in Fig. 5, the agent in state s will choose action a according to the policy $\pi(a|s)$. Then randomly transfer to the next state s' and obtain the reward $r(s, a)$ corresponding to the state transition $s \to s'$. As mentioned above, the optimization goal of the agent is to find the policy $\pi(a|s)$ that can maximize the long-term cumulative reward. Thus, this process can be described as

$$\max \ \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \, r(s, a)\right], \tag{20}$$

where the discount factor $0 < \gamma < 1$ represents the weight of each reward $r(s, a)$ obtained over time.

To facilitate learning and use, a series of value functions describe the above process (20). In particular, $V(s)$ is defined to represent the value of state s:

$$V(s) \triangleq \sum_{a \in A} \pi(a|s) \, Q(s, a), \tag{21}$$

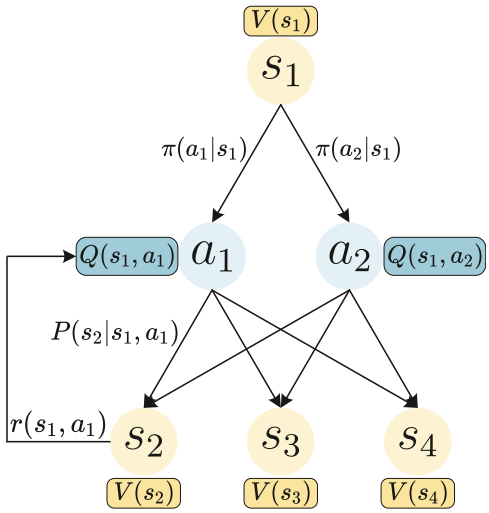


Fig. 5. Illustration of the Markov decision process.

where A is the action space; and $Q(s, a)$ is defined to represent the state-action value function:

$$Q(s, a) \triangleq r(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V(s'), \tag{22}$$

where S denotes all possible next states and $P(s'|s, a)$ is the state transition probability of $s \to s'$. If the state transition of the system is fixed (i.e., the MEC system under study), we have the following relationship

$$Q(s, a) = r(s, a) + \gamma V(s'). \tag{23}$$

The Bellman equation describing the optimal state-action value function $Q^*(s, a)$ in DDPG is expressed as [31]

$$Q^*(s, a) = r(s, a) + \gamma \max Q^*(s', a'). \tag{24}$$

Unlike (23), (24) updates $Q^*(s, a)$ through greedy strategy, which leads to the overestimation of the actual Q-function by the critic network. A similar problem also exists in the DQN algorithm.

The TD3 algorithm introduces the clipped double Q-learning technique to suppress overestimation. Specifically, TD3 initializes double critic networks $(Q_{\theta_1}, Q_{\theta_2})$ with the same structure but different parameters (θ_1, θ_2) to estimate the Q-function at the same time, and the smaller Q-function will be chosen as the actual one during the training process. Accordingly, the actual $Q_\theta(s, a)$ can be expressed as

$$Q_\theta(s, a) = \min(Q_{\theta_1}(s, a), Q_{\theta_2}(s, a)). \tag{25}$$

Although these critic networks have the same update goals, the estimation of the Q-function will be different due to the other initialization parameters (θ_1, θ_2). Therefore, the TD3 algorithm can effectively avoid the overestimation problem in DDPG.

2) Delayed Policy Update: In a standard critic-actor structure, the critic network is used to fit the Q-function, and the actor network is responsible for optimizing the action policy $\pi(a|s)$ to achieve the maximum cumulative reward. The problem is that the optimization goal of the actor network highly depends on how well the critic network fits the actual Q-function. In the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

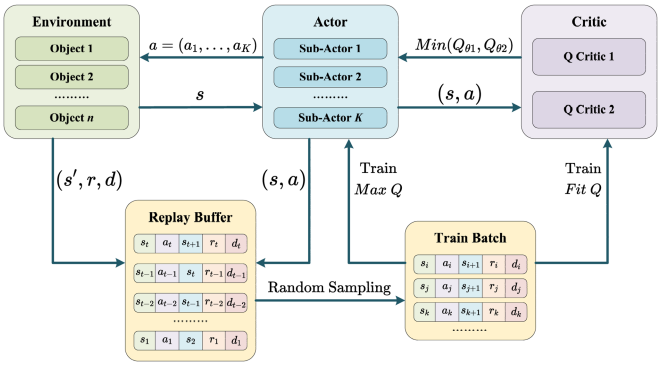


Fig. 6. Illustration of the proposed MAPTD3 agent.

training process, the actual Q-function to be fitted by the critic network is constantly changing, and there is a particular gap with the essential Q-function, especially in the early training stage, which will cause the actor network's optimization goal to be unclear, and further lead to unstable agent training. The TD3 algorithm adopts the delayed update policy to train the actor network to stabilize the training process. Specifically, the TD3 algorithm updates the actor network with a lower update frequency than the critic network, allowing actors to have more explicit optimization goals and thus stabilizing the training process.

3) Target Policy Smoothing: During the training process, by adding cropped noise to the action policy and clipping the target's action to the valid action range, the TD3 agent can better explore the environment and improve the generalization of the agent. Notice that the cropped noise is only added during training and is unnecessary in real-world applications [32].

D. MAPTD3 Algorithm

A standard TD3 algorithm has only one actor network in the critic-actor structure. However, for complex problems, such as NP-hard like adaptive joint resource allocation in high-dynamic IoV scenarios, using a single actor network to control multiple strategies will inevitably boost the network dimension to ensure performance, which will lead to an exponential increase in network space complexity, and the training process will become hard to converge. To address these problems, we propose a customized TD3 algorithm, say, multi-actor parallel TD3 (MAPTD3), which replaces the original single actor network with multiple parallel sub-actor networks, as shown in Fig. 6. By decoupling the complex joint optimization problem into multiple sub-problems and using multiple low-dimensional sub-actor networks to optimize complementary strategies, the exponentially growing space complexity is transformed into linear growth such that the space complexity of actor network can be effectively reduced [33].

In brief, the proposed MAPTD3 algorithm is summarized in Algorithm 2, where the update method of the critic network is consistent with the TD3 algorithm. The main difference lies in the update method of the actor network. Specifically, when updating the actor network, the TD3 algorithm will recalculate the Q-function to ensure that the latest one is used and then

Algorithm 2: Proposed MAPTD3 Algorithm.

Initialization: The random parameters (θ_1, θ_2) for double critic networks $(Q_{\theta 1}, Q_{\theta 2})$, the random parameters $\{\phi_i | i = 1, 2, \ldots, K\}$ for sub-actor networks $\{\pi_{\phi_i} | i = 1, 2, \ldots, K\}$, the target critic and sub-actor networks set $\theta_1' \leftarrow \theta_1$, $\theta_2' \leftarrow \theta_2$, $\phi_i' \leftarrow \phi_i$, the reply buffer parameter $\mathcal{B}$, the batch size L.
for $t = 1$ to T **do**
  Selection action with exploration noise $a_i \sim \pi_{\phi_i}(s) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma)$;
  $a \leftarrow (a_1, a_2, \ldots, a_i, \cdot, a_K)$;
  Execute action a and observe reward r, new state s';
  Store transition tuple (s, a, r, s') in $\mathcal{B}$;
  Sample training batch transitions (s, a, r, s') from $\mathcal{B}$;
  $a_i' \leftarrow \pi_{\phi_i'}(s') + \epsilon$, where $\epsilon \sim$ clip $(\mathcal{N}(0, \tilde{\sigma}), -c, c)$[1];
  $a' \leftarrow (a_1', a_2', \ldots, a_i', \ldots, a_K')$;
  $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta_i'}(s', a')$;
  Update critics $\theta_i \leftarrow \arg\min_{\theta_i} \frac{1}{L} \sum_{l \in L} (y - Q_{\theta_i}(s, a))^2$;
  if t mod $d = 0$ **then**
    $\tilde{a}_i \sim \pi_{\phi_i}(s) + \epsilon$;
    $\tilde{a} \leftarrow (\tilde{a}_1, \tilde{a}_2, \ldots, \tilde{a}_i, \ldots, \tilde{a}_K)$;
    Update ϕ_i by the deterministic policy gradient: $\nabla_{\phi_i} J(\phi_i) = \frac{1}{L} \sum_{l \in L} \nabla_{\tilde{a}_i} Q_{\theta_1}(s, \tilde{a}) \nabla_{\phi_i} \pi_{\phi_i}(s)$;
    Update target networks: $\theta_i' \leftarrow \tau \theta_i + (1 - \tau) \theta_i'$, $\phi_i' \leftarrow \tau \phi_i + (1 - \tau) \phi_i'$;
  end if
end for

perform gradient ascent on the derivative of the Q-function to update the network parameters. Mathematically, this process can be expressed as

$$\nabla_\phi J(\phi) = \frac{1}{L} \sum_{l \in L} \nabla_{\tilde{a}} Q_{\theta_1}(s_l, \tilde{a}) \Big|_{\tilde{a} = \pi_\phi(s_l)} \nabla_\phi \pi_\phi(s_l), \quad (26)$$

where L is the batch size. For the MAPTD3 algorithm, the Q-function must also be recalculated. The difference is that when deriving the Q-function, each sub-actor network only calculates the partial derivatives for related actions and performs gradient ascent on the corresponding results to update network parameters, expressed as

$$\nabla_{\phi_i} J(\phi_i) = \frac{1}{L} \sum_{l \in L} \nabla_{\tilde{a}_i} Q_{\theta_1}(s_l, \tilde{a}) \Big|_{\tilde{a}_i = \pi_{\phi_i}(s_l)} \nabla_{\phi_i} \pi_{\phi_i}(s_l). \quad (27)$$

E. Adaptive Resource Allocation Framework

In summary, Fig. 7 depicts the adaptive resource allocation framework developed in this paper. In this framework, the MEC system is abstracted into two queues: request buffer queue (say, Q_r) and task run queue (say, Q_t), as described in Subsection IV-B, and each task in the latter queue contains seven state parameters. Combined with the cache size limit of Q_t (i.e.,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

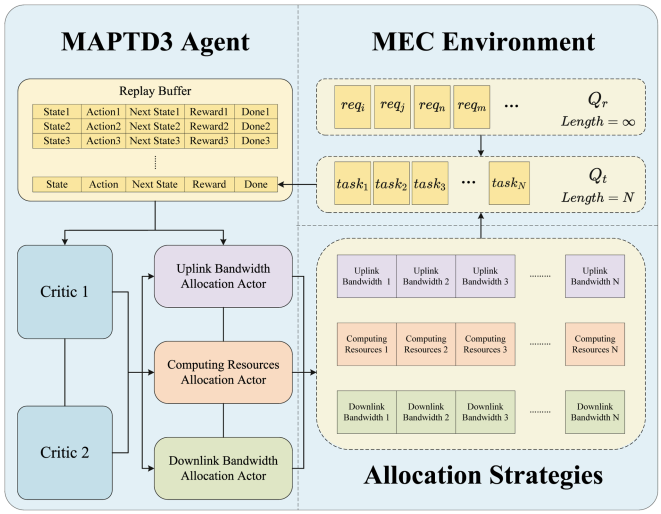


Fig. 7. Illustration of the adaptive resource allocation framework.

N), the dimension of the state space is finally determined as $(N, 7)$. The MAPTD3 agent runs on the edge server and contains three parallel sub-actor networks, which are used to output the system's uplink, computing, and downlink resource allocation strategies. Considering the impact of the softmax operation, the action space dimension is defined as $(N + 1, 3)$, implying three sub-actor networks are used for each output action of dimension $N + 1$.

V. Simulation Results and Discussions

In this section, we present and discuss the simulation results of the proposed adaptive joint resource allocation scheme. Firstly, the parameter setting for the pertaining simulation experiments is described. Then, the proposed scheme is compared with two traditional methods, a state-of-art machine learning method, and two DRL agents regarding convergence, effectiveness, complexity, and robustness. Finally, the impact of communication distance and task priority weights on the overall performance of the proposed scheme is analyzed.

A. Setup of Simulation Experiments

We performed simulation experiments on a MEC-enabled IoV system in a V2X cell with a radius of 500 m, while the BS equipped with MIMO antennas and edge server is located at the center of the cell. Vehicles are randomly distributed in the annulus of inner radius 25 m and outer radius 500 m, which means that there is an exclusion zone of radius 25 m around the BS to simulate the distance between the BS and the road, and all vehicles are in its far field [34]. For vehicle m, $task_m$ arrives randomly within $[0, 500]$ ms. The delay threshold, upload data size, number of CPU cycles required, and download data size are uniformly distributed in $[50, 500]$ ms, $[10, 30]$ Mbits, $[100, 500]$ Mcycles, $[5, 15]$ Mbits, respectively. The weight factor ratio $\theta^c : \theta^t : \theta^e$ is set to $10 : 5 : 1$, and the communication transmission is subject to the path fading model $128.1 + 37.6 \log_{10} d$, corresponding to the V2I model during the propagation process [34]. The other parameters are listed in Table II, following those widely used in the literature [5], [15], [26], [34].

TABLE II
Simulation Parameter Setting

Parameter	Value
Slot length (ms)	10
Simulation time (ms)	1000
Uplink bandwidth of BS (MHz)	10
Downlink bandwidth of BS (MHz)	5
Computing capacity of edge server (GHz)	25
Tx power of BS (dBm)	30
Tx power of vehicles (dBm)	27
Standby power of vehicles (dBm)	20
Noise power spectral density (dBm/Hz)	-174

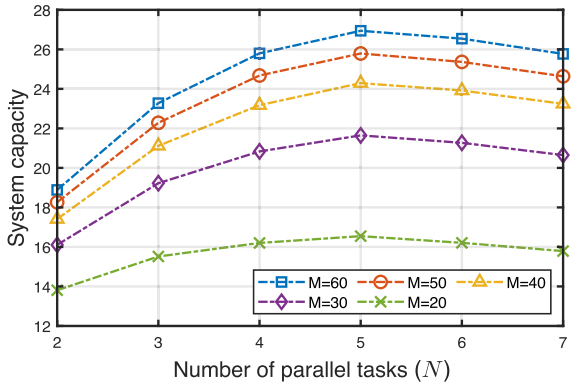


Fig. 8. System capacity versus the number of parallel tasks.

Moreover, considering that the total resources and the task model of the MEC system are determined, there inevitably exists an optimal number of parallel tasks N_{opt}. When $N < N_{opt}$, the weak parallel processing capability is the main factor limiting the system capacity, so the system capacity will increase with N. When $N > N_{opt}$, the limited resources will be unable to bear too many tasks parallel processing, which will greatly reduce the average resource available for each task, and finally lead to the system capacity decrease as N increases. Therefore, it is very necessary to obtain the optimal N_{opt} before the comparison experiment. Fig. 8 shows the trend of system capacity with N. It can be easily observed that $N = 5$ is the optimal solution that aligns with the aforementioned description.

All experiments in this paper were run on a workstation with AMD 3700X 8Core CPU, 32GB RAM, and NVIDIA RTX 3070 8G GPU. The corresponding software environment includes Anaconda3-5.3.1, Python 3.8, and TensorFlow 2.6. In the training phase, the number of steps in each episode is dynamically adjusted according to the completion time of all tasks, and the maximum number of steps in each episode is set to 100. For MAPTD3 agents, each training takes approximately 21 hours.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

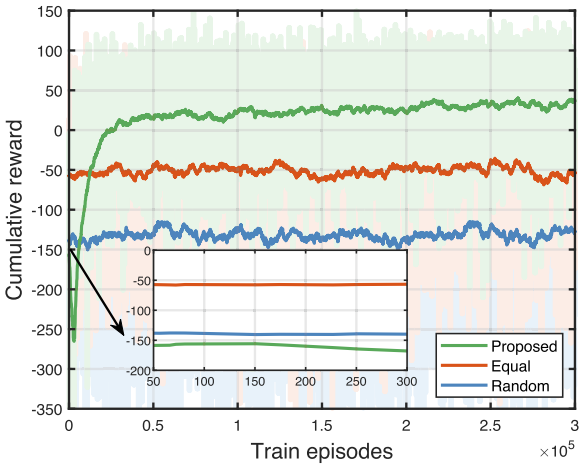


Fig. 9. Convergence of the proposed scheme during the training process.

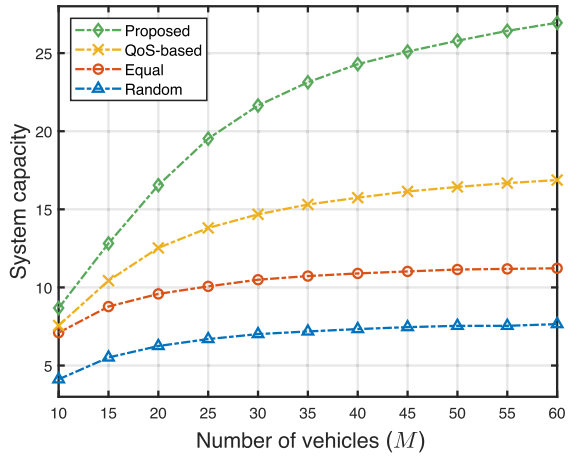


Fig. 10. System capacity versus the number of vehicles.

B. Convergence and Effectiveness

To evaluate the convergence and effectiveness of the proposed adaptive joint resource allocation scheme, we compare the proposed scheme with two typical baseline ones and a state-of-art machine learning baseline one:

1) *Random resource allocation:* It ignores any state of the current MEC system and randomly adjusts the resource allocation strategy during each time slot;
2) *Equal resource allocation:* It only considers the current number of running tasks in the MEC system and equally allocates the system resources according to the number of running tasks during each time slot;
3) *QoS-based resource allocation:* It considers all the current and future states of the MEC system and adaptively adjusts the resource allocation strategy according to the TD3 agent trained on the QoS-based reward function during each time slot [35];
4) *The proposed scheme:* It considers all the current and future states of the MEC system and adaptively adjusts the resource allocation strategy according to the MAPTD3 agent trained on the proposed reward function (18) during each time slot.

Fig. 9 compares the convergence performance of the two baseline and the proposed schemes in the training process, considering that the QoS-based scheme employ a different reward function, so the direct comparison is not conducted here. At the beginning of training, since the three sub-actor networks of the MAPTD3 agent are initialized with random parameters, the performance is similar to the Random scheme and lower than the Equal scheme. As the number of training steps increases, the cumulative reward of the proposed scheme grows rapidly and dramatically exceeds the Random and Equal schemes. When the number of training steps is sufficient, the cumulative reward of the proposed scheme will gradually and steadily converge to the optimal value. This verifies that the proposed scheme can obtain a stable adaptive resource allocation strategy from the specially designed training environment without prior knowledge or assumptions.

To further verify the effectiveness of the proposed scheme, this paper also compares the system performance of the four schemes in the simulation environment. Fig. 10 shows the system capacity performance of all schemes. It can be seen that the system capacity keeps rising with M for all methods. At the same time, compared with the three baseline schemes, the system capacity of the proposed scheme has been dramatically improved, and this advantage will be further expanded with the increase of M. Specifically, compared with the Random, Equal, and QoS-based schemes, the maximum system capacity improvement of the proposed method is 352%, 240%, and 160%, respectively. This is because the Random scheme ignores all states and Equal scheme only consider partial states while the proposed method optimizes this. Further, although the QoS-based scheme adjust the resource allocation strategy in real-time according to the all system state, the employed reward function only penalizes the agent when the QoS falls below a certain threshold. This design makes the reward of the agent with high QoS very sparse, which is insufficient to realize the optimal adaptive resource allocation strategy. In contrast, the proposed scheme comprehensively considers all aspects, such as task arrival time, data size, and vehicle distance, and defines the corresponding reward function for each time slot, thus achieving the best system performance.

Fig. 11 illustrates the average service delay with a different number of vehicles among the Random, Equal, QoS-based, and proposed schemes. The average service delay includes waiting, uploading, computing, and downloading delays. It can be observed from Fig. 11 that the average service delay of all schemes increases with M. This is because in a fixed time, a larger M means more tasks arrive in each time slot, which requires the resource-constrained MEC system to spend more time processing these tasks, and finally leads to a longer time for the MEC system to respond to other task offloading requests. Specifically, with the increase of M, the proportion of tasks waiting for delays increases significantly in all schemes. In addition, the proposed scheme benefits from the more flexible resource allocation strategy, so it performs a massive advantage over other schemes in all scenarios.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

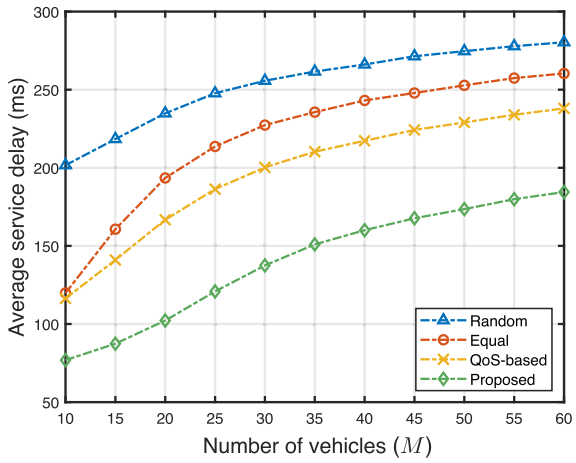


Fig. 11. Average service delay versus the number of vehicles.

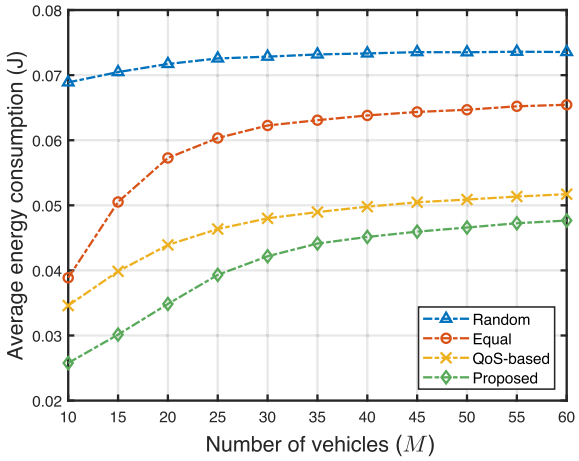


Fig. 12. Average energy consumption versus the number of vehicles.

Fig. 12 compares the average energy consumption of the four different schemes under study. The energy consumption includes signal transmission and standby power energy, and the signal transmit power is much higher than the standby power. The figure shows that the average energy consumption of Random, Equal, QoS-based, and the proposed schemes increases with M. This is because the addition of M will also reduce the communication resources available for each task, which makes the tasks need more time for data transmission, leading to an increase in energy consumption. Moreover, even though the proposed scheme assigns lowest weights to energy consumption than other metrics, it can still achieve optimal performance among the four schemes.

C. Robustness and Complexity

We compare the following DRL agents under the proposed reward function to evaluate the performance of the MAPTD3 agent and the commonly used DRL agents regarding robustness and complexity.

1) DDPG agent: The traditional DDPG agent for addressing the adaptive resource allocation problem [23];

TABLE III
COMPLEXITY OF DIFFERENT AGENTS

	Agent	Input*	Output*	Actor network structure			Complexity	Proportion**
				Neurons	Layer	Amount		
L	DDPG/TD3	35	18	16	3	1	1,426	102.2%
	MAPTD3	35	6	8	3	3	1,458	
M	DDPG/TD3	35	18	64	3	1	11,794	88.1%
	MAPTD3	35	6	32	3	3	10,386	
H	DDPG/TD3	35	18	256	3	1	145,426	79.2%
	MAPTD3	35	6	128	3	3	115,218	

* Calculated by the state space dimension $(N, 7)$ and the action space dimension $(N + 1, 3)$ described in Subsection IV-E, where $N = 5$.
** The complexity ratio of the MAPTD3 agent to the DDPG/TD3 agent.

2) TD3 agent: Unlike the DDPG agent, overestimation and weak robustness problems are avoided by using double critic networks and delayed policy update [30];
3) MAPTD3 agent: Unlike the TD3 agent, multiple low-dimensional sub-actor networks replace the original single high-dimensional actor network.

This experiment sets the learning rate to $1e-3$ for all critic and $3e-4$ for all actor networks. Note that the critic network is not involved in the decision-making process; all networks adopt the fully connected structure. The complexity to be discussed below refers specifically to the space complexity of the actor network. To make the experimental results universal, this paper sets up three groups of experiments with low complexity (L), medium complexity (M), and high complexity (H), and the specific parameters are listed in Table III. After repeated testing, each set of experiments can ensure that the decision time is less than the length of a one-time slot.

Figs. 13, 14, and 15 illustrate the performance of different DRL agents in terms of robustness, system capacity, service delay, and energy consumption under low, medium, and high complexity conditions, respectively. Obviously, the system performance keeps growing with the increased space complexity for all agents. Furthermore, as the complexity increases, TD3 and the proposed MAPTD3 agent show more significant performance gains than the DDPG agent. The relevant reason can be found in Figs. 13(a), 14(a), and 15(a). As shown, the DDPG agent exhibits the worst robustness, and its robustness keeps dropping with the increase of complexity, leading to the strategy's unstable convergence. It ultimately causes little improvement under medium and high complexity conditions. In contrast, the TD3 and MAPTD3 agents are more robust in all conditions, demonstrating that the double critic network can effectively stabilize the training process.

Further, it is shown in Figs. 13(b), 14(b), and 15(b) that due to the unstable strategy, the system capacity of the DDPG agent is significantly lower than that of TD3 and MAPTD3 agents. In addition, although the proposed MAPTD3 agent adopts a lower complexity, the system outperforms the other two agents. Figs. 13(c), 14(c), and 15(c) show the average service delay performance versus the number of vehicles among these agents. These figures demonstrate that the service delay of the proposed MAPTD3 agent is comparable to that of DDPG and TD3 agents in most experiments. However, in some low-complexity experiments, the MAPTD3 agent exhibits a slightly longer service delay, while in other experiments, it can achieve a similar service

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

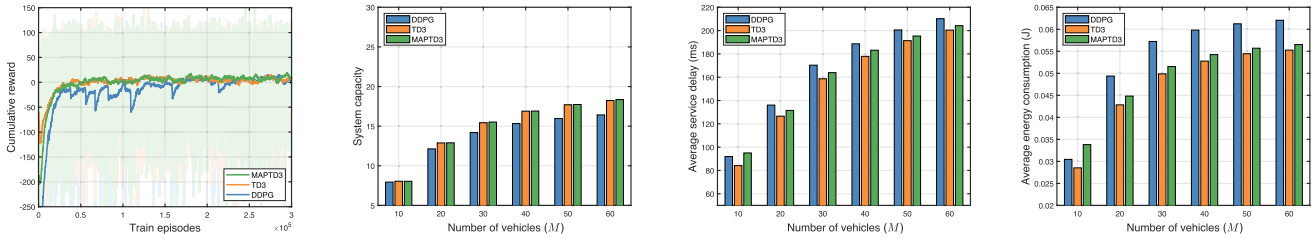


Fig. 13. Performance of DDPG, TD3 and MAPTD3 agents under low complexity. (a) Cumulative reward. (b) System capacity. (c) Service delay. (d) Energy consumption.

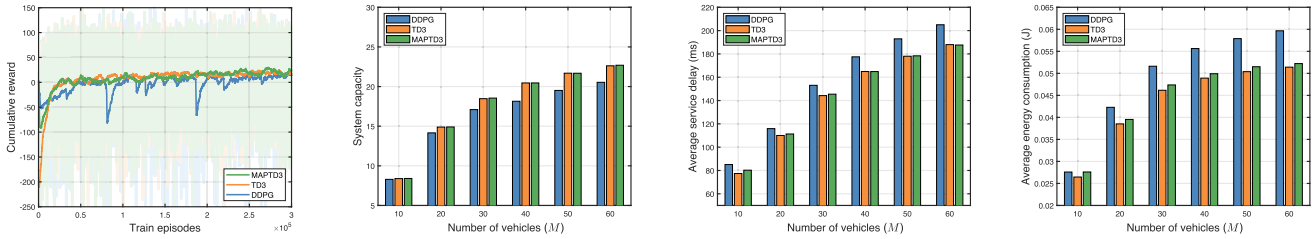


Fig. 14. Performance of DDPG, TD3 and MAPTD3 agents under medium complexity. (a) Cumulative reward. (b) System capacity. (c) Service delay. (d) Energy consumption

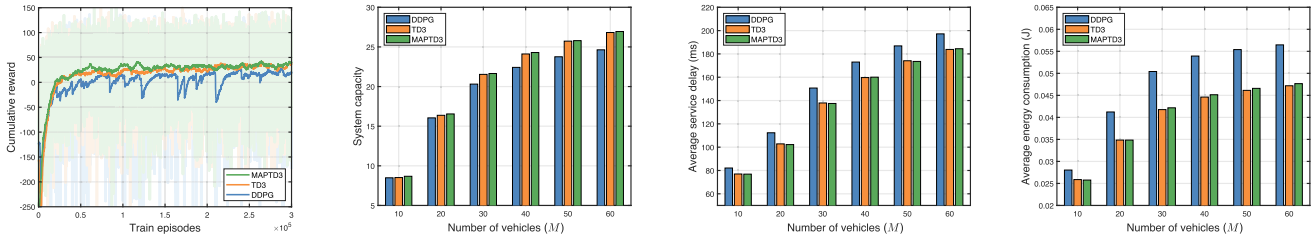


Fig. 15. Performance of DDPG, TD3 and MAPTD3 agents under high complexity. (a) Cumulative reward. (b) System capacity. (c) Service delay. (d) Energy consumption

delay as TD3 agent and much lower than DDPG agent. The same phenomenon is also reflected in energy consumption. One reason for this phenomenon is that the MAPTD3 agent uses sub-actor networks with identical structures to control different resource allocation strategies while ignoring the potential difference in optimization difficulty among these strategies. As a result, some low-complexity sub-actor networks may struggle to fully capture all the relevant information in the state, leading to suboptimal optimization of secondary system performance metrics such as delay and energy. Overall, the above results verify that the proposed MAPTD3 agent can effectively reduce the scheme's complexity while ensuring system performance, and this advantage will expand with the complexity.

D. Overall Performance Evaluation

To demonstrate the overall performance of the proposed resource allocation scheme, we depict the evolution of the task completion probability and average service delay under different combinations of vehicle distance, task priority weight, and delay threshold in Figs. 16 and 17, respectively. In this experiment, the priority weight of tasks is divided into five levels ranging from low to high and randomly initialized according to the probability of 0.4, 0.2, 0.2, 0.1 and 0.1, respectively. At the same time, the distance of vehicles and task delay thresholds are randomly initialized based on a uniform distribution within the range set above.

Fig. 16 shows the impact of the delay threshold and distance combination on the overall performance. From the upper panel of Fig. 16, it is observed that the task completion probability keeps decreasing with increasing vehicle distance for a fixed task delay threshold. This is because, according to the V2I path fading model, the increase of the distance will reduce the received power intensity of both communication parties, which makes the vehicle spend more time for data transmission when the same resources are allocated, and eventually lead to the decrease of

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

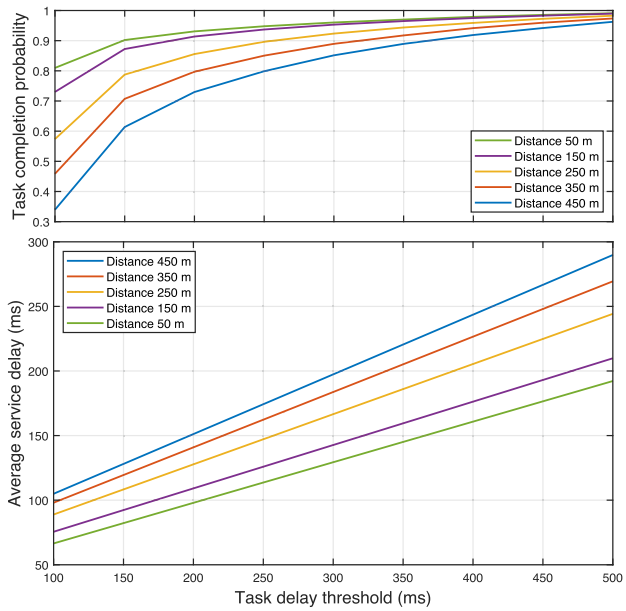


Fig. 16. Task completion probability/average service delay versus vehicles' distance and task delay threshold.

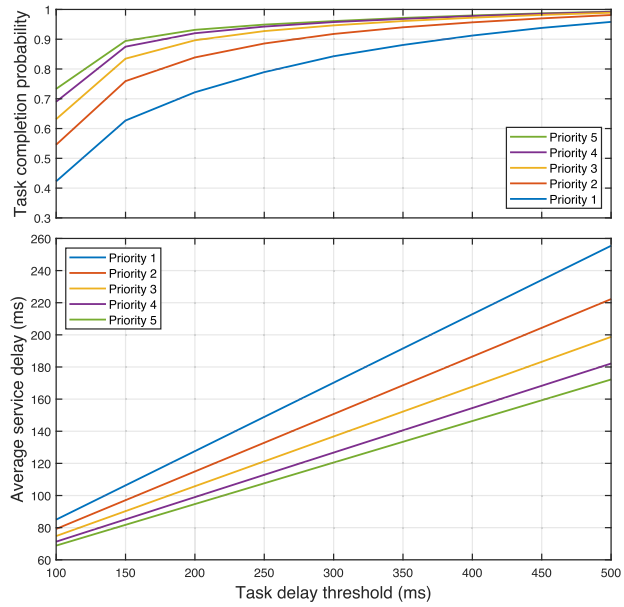


Fig. 17. Task completion probability/average service delay versus priority and task delay threshold.

the completion probability. The above conclusion can also be verified in the average delay performance (cf. the lower panel of Fig. 16). In addition, the rise in the task delay threshold means that the scheme has more free time slots to schedule the system resource reasonably, so that the completion probability can be effectively improved under a high task delay threshold. However, the above completion probability improvement comes at the cost of higher service delay. Specifically, it can be seen from Fig. 16 that the average service delay will gradually increase with the delay threshold increase.

The impact of the task priority weight and task delay threshold combination on the overall performance is shown in Fig. 17. The figure shows that the high-priority task has a higher task completion probability and lower average service delay for all delay thresholds. The reason is that the task offloading request generated by the high-priority task will be preferentially responded to by the dynamic task queue algorithm, which effectively reduces the waiting delay of the task. At the same time, the high-priority task will generate more vital feedback for each task slice, forcing the resource allocation scheme to unfairly allocate more system resources to the task to complete it as soon as possible.

In summary, the simulation results corroborate the effectiveness of the proposed adaptive multi-resource joint allocation scheme. Especially in the scenario with a large number of vehicles, the proposed method can effectively improve the comprehensive performance of the MEC system in terms of capacity, delay, and energy consumption. Moreover, the proposed MAPTD3 agent provides an efficient and robust optimization method by using multiple low-dimensional sub-actor networks and double critic networks, which significantly reduces the space complexity of the proposed resource allocation scheme while ensuring the system's performance.

VI. Conclusion

This paper performed adaptive multi-resource joint allocation in the MEC system with multiple vehicles to optimize the system performance. The developed scheme includes uplink, computing, and downlink resource allocation strategies and an adaptive resource allocation framework supporting dynamic task model. In addition, the MAPTD3 agent has been offered to optimize these strategies, which can effectively improve system performance and reduce the complexity of above resource allocation strategies. By modeling the state space and reward function in a unique way named task-slicing, the MAPTD3 agent does not need to pre-understand the statistical characteristics of the environment during the training process. The numerical results demonstrate that the proposed scheme can achieve close-to-optimal system performance with the lowest complexity while satisfying the service delay and system resource constraints. In future work, we will investigate the MEC system with multiple cells and introduce technologies such as channel estimation to overcome problems caused by vehicle mobility.

References

[1] S. Wan, R. Gu, T. Umer, K. Salah, and X. Xu, "Toward offloading Internet of Vehicles applications in 5G networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 7, pp. 4151–4159, Jul. 2021.

[2] J. Zhao, J. Liu, L. Yang, B. Ai, and S. Ni, "Future 5G-oriented system for urban rail transit: Opportunities and challenges," *IEEE China Commun.*, vol. 18, no. 2, pp. 1–12, 2021.

[3] C.-F. Liu, M. Bennis, M. Debbah, and H. V. Poor, "Dynamic task offloading and resource allocation for ultra-reliable low-latency edge computing," *IEEE Trans. Commun.*, vol. 67, no. 6, pp. 4132–4150, Jun. 2019.

[4] J. Zhao, S. Ni, L. Yang, Z. Zhang, Y. Gong, and X. You, "Multiband cooperation for 5G HetNets: A promising network paradigm," *IEEE Veh. Technol. Mag.*, vol. 14, no. 4, pp. 85–93, Dec. 2019.

[5] J. Chen, H. Xing, Z. Xiao, L. Xu, and T. Tao, "A DRL agent for jointly optimizing computation offloading and resource allocation in MEC," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17508–17524, Dec. 2021.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.

[6] F. Jiang, K. Wang, L. Dong, C. Pan, W. Xu, and K. Yang, "Deep-learning-based joint resource scheduling algorithms for hybrid MEC networks," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6252–6265, Jul. 2020.
[7] G. Miao, J. Zander, K. W. Sung, and S. B. Slimane, *Fundamentals of Mobile Data Networks*. Cambridge, U.K.: Cambridge Univ. Press, 2016.
[8] B. Liu, C. Liu, and M. Peng, "Resource allocation for energy-efficient MEC in NOMA-Enabled massive IoT networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 4, pp. 1015–1027, Apr. 2021.
[9] J. Zhao, Q. Li, Y. Gong, and K. Zhang, "Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, pp. 7944–7956, Aug. 2019.
[10] J. Zhang, W. Xia, F. Yan, and L. Shen, "Joint computation offloading and resource allocation optimization in heterogeneous networks with mobile edge computing," *IEEE Access*, vol. 6, pp. 19324–19337, 2018.
[11] L. Huang, X. Feng, A. Feng, Y. Huang, and L. P. Qian, "Distributed deep learning-based offloading for mobile edge computing networks," *Mobile Netw. Appl.*, vol. 27, pp. 1123–1130, 2022.
[12] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11158–11168, Nov. 2019.
[13] J. Zhao, Y. Zhang, Y. Nie, and J. Liu, "Intelligent resource allocation for train-to-train communication: A multi-agent deep reinforcement learning approach," *IEEE Access*, vol. 8, pp. 8032–8040, 2020.
[14] H. Peng and X. Shen, "Multi-agent reinforcement learning based resource management in MEC-and UAV-Assisted vehicular networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 1, pp. 131–141, Jan. 2021.
[15] J. Zhao, X. Sun, Q. Li, and X. Ma, "Edge caching and computation management for real-time Internet of Vehicles: An online and distributed approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 4, pp. 2183–2197, Apr. 2021.
[16] Z. Chang, L. Liu, X. Guo, and Q. Sheng, "Dynamic resource allocation and computation offloading for IoT fog computing system," *IEEE Trans. Ind. Inform.*, vol. 17, no. 5, pp. 3348–3357, May 2021.
[17] J. Zhao, L. Yang, M. Xia, and M. Motani, "Unified analysis of coordinated multipoint transmissions in mmWave cellular networks," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12166–12180, Jul. 2022.
[18] Z. Tong, J. Cai, J. Mei, K. Li, and K. Li, "Dynamic energy-saving offloading strategy guided by Lyapunov optimization for IoT devices," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19903–19915, Oct. 2022.
[19] L. Liu, X. Qin, Z. Zhang, and P. Zhang, "Joint task offloading and resource allocation for obtaining fresh status updates in multi-device MEC systems," *IEEE Access*, vol. 8, pp. 38248–38261, 2020.
[20] L. Wang et al., "RL-based user association and resource allocation for Multi-UAV enabled MEC," in *Proc. IEEE 15th Int. Wireless Commun. Mobile Comput. Conf.*, 2019, pp. 741–746.
[21] J. Li, H. Gao, T. Lv, and Y. Lu, "Deep reinforcement learning based computation offloading and resource allocation for MEC," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2018, pp. 1–6.
[22] Z. Tong, X. Deng, F. Ye, S. Basodi, X. Xiao, and Y. Pan, "Adaptive computation offloading and resource allocation strategy in a mobile edge computing environment," *Inf. Sci.*, vol. 537, pp. 116–131, 2020.
[23] T. Song et al., "DRAM: A. DRL-based resource allocation scheme for MAR in MEC," *Digit. Commun. Netw.*, vol. 9, no. 3, pp. 723–733, 2023.
[24] Y. Gong, J. Wang, and H. Yao, "Distributed multi-agent empowered resource allocation in deep edge networks," in *Proc. IEEE Int. Wireless Commun. Mobile Comput.*, 2021, pp. 974–979.
[25] G. Matheron, N. Perrin, and O. Sigaud, "Understanding failures of deterministic actor-critic with continuous action spaces and sparse rewards," in *Proc. Int. Conf. Artif. Neural Netw. Mach. Learn.*, 2020, pp. 308–320.
[26] N. Waqar, S. A. Hassan, A. Mahmood, K. Dev, D.-T. Do, and M. Gidlund, "Computation offloading and resource allocation in MEC-Enabled integrated aerial-terrestrial vehicular networks: A reinforcement learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21478–21491, Nov. 2022.
[27] E. B. Dynkin, *Markov Processes*. Berlin, Germany: Springer, 1965, pp. 77–104.
[28] S. Sharma, S. Sharma, and A. Athaiya, "Activation functions in neural networks," *Towards Data Sci.*, vol. 6, no. 12, pp. 310–316, 2017.
[29] M. Riedmiller et al., "Learning by playing solving sparse reward tasks from scratch," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 4344–4353.
[30] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
[31] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016.
[32] J. Achiam, "Spinning up in deep reinforcement learning," 2018. [Online]. Available: https://spinningup.openai.com/en/latest/algorithms/td3.html
[33] X. Yang, J. Ye, and X. Wang, "Factorizing knowledge in neural networks," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 73–91.
[34] L. Liang, G. Y. Li, and W. Xu, "Resource allocation for D2D-Enabled vehicular communications," *IEEE Trans. Commun.*, vol. 65, no. 7, pp. 3186–3197, Jul. 2017.
[35] J. Huang, J. Wan, B. Lv, Q. Ye, and Y. Chen, "Joint computation offloading and resource allocation for edge-cloud collaboration in internet of vehicles via deep reinforcement learning," *IEEE Syst. J.*, vol. 17, no. 2, pp. 2500–2511, Jun. 2023.



Junhui Zhao (Senior Member, IEEE) received the M.S. and Ph.D. degrees from Southeast University, Nanjing, China, in 1998 and 2004, respectively. From 1998 to 1999, he was with the Nanjing Institute of Engineers, ZTE Corporation, Shenzhen, China. In 2004, he was an Assistant Professor with the Faculty of Information Technology, Macao University of Science and Technology, Cotai, China, and continued there as an Associate Professor till 2007. In 2008, he was with Beijing Jiaotong University as an Associate Professor, where he is currently a Professor with the School of Electronics and Information Engineering. Since 2016, he has also been with the School of Information Engineering, East China Jiaotong University, Nanchang, China. Meanwhile, he was also a short term Visiting Scholar with Yonsei University, Seoul, South Korea, in 2004 and a Visiting Scholar with Nanyang Technological University, Singapore from 2013 to 2014. His research interests include wireless and mobile communications and related applications.



Haoyu Quan received the B.S. degree in communication engineering from East China Jiaotong University, Jiangxi, China, in 2021. He is currently working toward the master's degree in information and communication engineering with East China Jiaotong University. His research interests include vehicular networks, mobile edge computing, resource allocation, and deep reinforcement learning techniques.



Minghua Xia (Senior Member, IEEE) received the Ph.D. degree in telecommunications and in-formation cystems from Sun Yat-sen University, Guangzhou, China, in 2007. From 2007 to 2009, he was with the Electronics and Telecommunications Research Institute, Daejeon, South Korea and Beijing R&D Center, Beijing, China, where he was a Member and then as a Senior Member with the engineering staff. From 2010 to 2014, he was in sequence with The University of Hong Kong, Hong Kong, King Abdullah University of Science and Technology, Jeddah, Saudi Arabia, and the Institut National de la Recherche Scientifique, University of Quebec, Montreal, QC, Canada, as a Postdoctoral Fellow. Since 2015, he has been a Professor with Sun Yat-sen University. Since 2019, he has also been an Adjunct Professor with the Southern Marine Science and Engineering Guangdong Laboratory, Zhuhai, China. His research interests include the general areas of wireless communications and signal processing.



Dongming Wang (Member, IEEE) received the B.S. degree from the Chongqing University of Posts and Telecommunications, Chongqing, China, in 1999, the M.S. degree from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2002, and the Ph.D. degree from Southeast University, Nanjing, in 2006. In 2006, he was with the National Mobile Communications Research Laboratory, Southeast University, where he is a Professor. His research interests include signal processing for wireless communications and large-scale distributed MIMO systems (cell-free massive MIMO).

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:18:27 UTC from IEEE Xplore. Restrictions apply.